

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Dot .Net Core and Progressive Web Application Practical

Practical – III

Roll No.:T009	Name:Siddhesh Kamble
Class:TYIT	Batch:01
Date of Assignment:14/07/2026	Date/Time of Submission:14/07/2026

Part A - Theory :

C# Delegate

What is a Delegate?

A **Delegate** is a special type in C# that stores the reference (address) of a method. It allows a method to be called indirectly through a delegate object. A delegate can call any method that has the **same return type and parameters** as the delegate declaration.

Why are Delegates Used?

- To call methods indirectly.
- To increase code flexibility.
- To implement callbacks.
- Used in event handling and GUI programming.

Real-Life Example

Think of a **TV Remote**.

- TV = Method
- Remote = Delegate

Instead of pressing buttons on the TV, we use the remote. Similarly, instead of calling a method directly, we use a delegate.

Syntax of Delegate

```
delegate returnType DelegateName(parameter_list);
```

Example

```
delegate void Operation(int a, int b);
```

Steps to Use a Delegate**Step 1: Declare the delegate**

```
delegate void Operation(int a, int b);
```

Step 2: Create methods having the same signature

```
public void Add(int a, int b)
{
    Console.WriteLine(a + b);
}
```

Step 3: Create a delegate object

```
Operation obj;
```

Step 4: Assign a method to the delegate

```
obj = Add;
```

Step 5: Invoke the delegate

```
obj(10, 20);
```

```
or
```

```
obj.Invoke(10, 20);
```

Part B - Program

- a) Create methods `add()`, `multiply()`, `subtract()`, `divide()` with suitable parameters and call these methods using concept of C# delegate.

Algorithm:

Step 1: Start the program

Step 2: Import System namespace

Step 3: Declare delegate `Operation(int a, int b)`

Step 4: Define class `Calculator`

Step 5: Create methods `Add`, `Sub`, `Multiply`, `Division`

Step 6: Define `Main()` method

Step 7: Create object of `Calculator`

Step 8: Declare delegate variable `obj`

Step 9: Assign each method to delegate and call it with values (10, 5)

Step 10: End the program

Code:

```
using System;

delegate void Operation(int a, int b);
class Calculator
{
    public void Add(int a, int b)
    {
        Console.WriteLine("Addition=" + (a + b));
    }
    public void Sub(int a, int b)
    {
        Console.WriteLine("Subtraction=" + (a - b));
    }
    public void Multiply(int a, int b)
    {
        Console.WriteLine("Multiply=" + (a * b));
    }
    public void Division(int a, int b)
    {
        Console.WriteLine("Division=" + (a / b));
    }
}
class Program
{
    static void Main()
    {
        Calculator c = new Calculator();
        Operation obj;
        obj = c.Add;
```

```
obj(10, 5);  
obj = c.Sub;  
obj(10, 5);  
obj = c.Multiply;  
obj(10, 5);  
obj = c.Division;  
obj(10, 5);  
Console.WriteLine("Siddhesh K (T009)");  
}  
}
```

Output:

```
Addition=15  
Subtraction=5  
Multiply=50  
Division=2  
Siddhesh K (T009)
```

b) Write a program using multicast delegates.

Algorithm:

- Step 1:** Start the program
- Step 2:** Import System namespace
- Step 3:** Declare delegate Message()
- Step 4:** Define class Demo
- Step 5:** Create methods Welcome, Learn, and ThankYou
- Step 6:** Define Main() method
- Step 7:** Create object of class Demo
- Step 8:** Declare delegate variable obj
- Step 9:** Assign methods using = and += and call delegate
- Step 10:** End the program

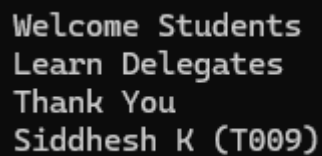
Code:

```
using System;  
  
delegate void Message();  
class Demo  
{  
    public void Welcome()  
    {  
        Console.WriteLine("Welcome Students");  
    }  
    public void Learn()  
    {  
        Console.WriteLine("Learn Delegates");  
    }  
    public void ThankYou()  
    {  

```

```
        Console.WriteLine("Thank You");
    }
}
class Program
{
    static void Main()
    {
        Demo d = new Demo();
        Message obj;
        obj = d.Welcome;
        obj += d.Learn;
        obj += d.ThankYou;
        obj();
        Console.WriteLine("Siddhesh K (T009)");
    }
}
```

Output:



```
Welcome Students
Learn Delegates
Thank You
Siddhesh K (T009)
```

- c) Create a class BankAccount with AccountNumber and Balance. Implement property for Balance with validation (Balance cannot be negative).

Algorithm:

- Step 1: Start the program
- Step 2: Import System namespace
- Step 3: Define class BankAccount
- Step 4: Declare property AccountNumber
- Step 5: Declare private variable balance
- Step 6: Create property Balance with getter and setter
- Step 7: In setter, check if value ≥ 0 , otherwise show error message
- Step 8: Define Main() method
- Step 9: Create object, assign values, and display account details
- Step 10: End the program

Code:

```
using System;

class BankAccount
{
    public int AccountNumber { get; set; }
    private double balance
    public double Balance
    {
```

```
        get
        {
            return balance;
        }
        set
        {
            if (value >= 0)
                balance = value;
            else
                Console.WriteLine("Balance cannot be negative.");
        }
    }
}
class Program
{
    static void Main()
    {
        BankAccount b = new BankAccount();
        b.AccountNumber = 101;
        b.Balance = 5000;
        Console.WriteLine("Account Number : " + b.AccountNumber);
        Console.WriteLine("Balance : " + b.Balance);
        b.Balance = -1000;
        Console.WriteLine("Siddhesh K (T009)");
    }
}
```

Output:

```
Account Number : 101
Balance : 5000
Balance cannot be negative.
Siddhesh K (T009)
```

Part C – Curiosity Question

1. A college wants to calculate student performance.

- a. Create methods to
- b. Calculate Total Marks
- c. Calculate Percentage
- d. Display Grade
- e. Display Pass/Fail

Note: Call each method using a delegate.

Algorithms :

Step 1: Start the program

Step 2: Import System namespace

Step 3: Declare delegate StudentDelegate()

Step 4: Define class Student

Step 5: Declare variables total and percentage

Step 6: Create methods to calculate total, percentage, grade, and result

Step 7: Define Main() method

Step 8: Create object of Student

Step 9: Assign methods to delegate using = and += and call delegate

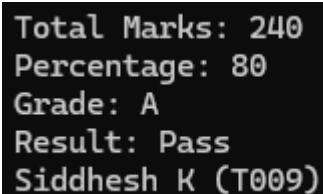
Step 10: End the program

Code :

```
using System;
delegate void StudentDelegate();
class Student
{
    int total;
    double percentage;
    public void CalculateTotal()
    {
        int m1 = 70, m2 = 80, m3 = 90;
        total = m1 + m2 + m3;
        Console.WriteLine("Total Marks: " + total);
    }
    public void CalculatePercentage()
    {
        percentage = total / 3.0;
        Console.WriteLine("Percentage: " + percentage);
    }
    public void DisplayGrade()
    {
        if (percentage >= 75)
            Console.WriteLine("Grade: A");
        else if (percentage >= 60)
            Console.WriteLine("Grade: B");
        else if (percentage >= 50)
            Console.WriteLine("Grade: C");
        else
            Console.WriteLine("Grade: Fail");
    }
}
```

```
}  
public void DisplayResult()  
{  
    if (percentage >= 50)  
        Console.WriteLine("Result: Pass");  
    else  
        Console.WriteLine("Result: Fail");  
}  
}  
class Program  
{  
    static void Main()  
    {  
        Student s = new Student();  
        StudentDelegate obj;  
        obj = s.CalculateTotal;  
        obj += s.CalculatePercentage;  
        obj += s.DisplayGrade;  
        obj += s.DisplayResult;  
        obj();  
        Console.WriteLine("Siddhesh K (T009)");  
    }  
}
```

Output :



```
Total Marks: 240  
Percentage: 80  
Grade: A  
Result: Pass  
Siddhesh K (T009)
```

2. A college wants to automate the admission process.
When the user clicks **Confirm Admission**, several operations must happen automatically. Design a delegate-based solution.

Students decide:

- a. Verify Documents
- b. Generate Roll Number
- c. Allocate Division
- d. Send SMS

Note: For the above operation just print a statement.

Algorithm :

- Step 1:** Start the program
- Step 2:** Import System namespace
- Step 3:** Declare delegate AdmissionDelegate()
- Step 4:** Define class Admission
- Step 5:** Create methods for document verification, roll number generation, division allocation, and SMS sending

Step 6: Define Main() method

Step 7: Create object of Admission

Step 8: Declare delegate variable obj

Step 9: Assign methods using = and += and call delegate

Step 10: End the program

Code :

```
using System;

delegate void AdmissionDelegate();
class Admission
{
    public void VerifyDocuments()
    {
        Console.WriteLine("Documents Verified");
    }
    public void GenerateRollNumber()
    {
        Console.WriteLine("Roll Number Generated");
    }
    public void AllocateDivision()
    {
        Console.WriteLine("Division Allocated");
    }
    public void SendSMS()
    {
        Console.WriteLine("SMS Sent to Student");
    }
}
class Program
{
    static void Main()
    {
        Admission a = new Admission();
        AdmissionDelegate obj;
        obj = a.VerifyDocuments;
        obj += a.GenerateRollNumber;
        obj += a.AllocateDivision;
        obj += a.SendSMS;
        Console.WriteLine("Confirm Admission Clicked...\n");
        obj();
        Console.WriteLine("\nSiddhesh K (T009)");
    }
}
```

Output :

Confirm Admission Clicked...

Documents Verified

Roll Number Generated

Division Allocated

SMS Sent to Student

Siddhesh K (T009)